Step 5 — The Full Team: a Supervisor Routing 4 Specialists via Command — Solution

> Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 5: four specialists, each proven alone. Now a **Supervisor** runs the team: it reads the shared state, decides who works next, logs why, and routes with LangGraph's `Command` (Doc11). The Writer ↔ Reviewer loop moves from `main.py` into the graph — same limit, same pass mark. The agents' inner `run_...()` functions don't change; each only gets a small node wrapper. **If not:** the order of work would stay hard-coded in `main.py`, with no shared state to inspect and no routing log to explain a bad run.

After this step the project is complete. Files not shown here don't change.

Solution — a testable routing function, a reason for every move, one writer per field

Story — `state.py`: the shared state, with **one writer per field** (Doc11's rule) — Research writes the notes, Analysis the findings, and so on; only the Supervisor writes the routing log. `research_done` records that Research ran even if it found nothing, so it's never repeated. **If not:** one agent's working notes could overwrite another's, and the Supervisor couldn't tell "not run yet" from "ran, found nothing".

```
# src/contentforge/state.py
from typing import TypedDict

class TeamState(TypedDict):
    # shared input — every agent may read it
    topic: str
    # why: each field has exactly ONE writer (Doc11's shared-state rule),
    # so no agent's working notes overwrite another's
    research_notes: str       # written by Research
    research_done: bool       # written by Research
    findings: str             # written by Analysis
    draft: str                # written by Writer (cleared by Reviewer)
    feedback_history: list[str]  # written by Reviewer
    accepted: bool            # written by Reviewer
    revision_count: int       # written by Reviewer
```

```
    routing_log: list[dict]   # written by the Supervisor only

def start_state(topic: str) -> TeamState:
    return {
        "topic": topic,
        "research_notes": "",
        "research_done": False,
        "findings": "",
        "draft": "",
        "feedback_history": [],
        "accepted": False,
        "revision_count": 0,
        "routing_log": [],
    }
```

Story — `agents/supervisor.py`: `decide_next()` is a plain function of the state that returns `(next step, reason)` — testable with a hand-made dict. `supervisor()` logs the decision and returns `Command(update=..., goto=...)`. The order of the rules is the whole routing policy: research → stop if nothing found → analysis → stop if accepted or at the limit → writer → reviewer. **If not:** routing bugs could only be found by running the whole team.

```
# src/contentforge/agents/supervisor.py
from langgraph.graph import END
from langgraph.types import Command

from contentforge.state import TeamState
from contentforge.utils.logger import get_logger

# why: the hard limit on Writer <-> Reviewer rounds — never loop forever
MAX_REVISIONS = 3

def decide_next(state: TeamState) -> tuple[str, str]:
    # why: a plain function of the state, returning (next step, reason) —
    # testable with a hand-made dict, no model and no graph needed
    if not state["research_done"]:
        return "research_agent", "no research yet"
    if state["research_notes"] == "":
        return "END", "research found no facts for this topic"
    if state["findings"] == "":
        return "analysis_agent", "research done, no findings yet"
    if state["accepted"]:
        return "END", "reviewer accepted the draft"
    if state["revision_count"] >= MAX_REVISIONS:
        return "END", f"revision limit ({MAX_REVISIONS}) reached"
    if state["draft"] == "":
        if state["feedback_history"]:
```

```
            return "writer_agent", "revising after reviewer feedback"
        return "writer_agent", "findings ready, no draft yet"
    return "reviewer_agent", "draft ready for review"

def supervisor(state: TeamState) -> Command:
    logger = get_logger(__name__)
    step, reason = decide_next(state)
    logger.info("supervisor -> %s (%s)", step, reason)
    # why: every decision is logged WITH its reason (Doc11), not only the
    # node name
    log = state["routing_log"] + [{"step": step, "reason": reason}]
    if step == "END":
        return Command(update={"routing_log": log}, goto=END)
    return Command(update={"routing_log": log}, goto=step)
```

Changed from Steps 2-4: each agent file gets a small **node wrapper** that reads its inputs from the state, calls the unchanged `run_...()` function, and returns `Command(update=..., goto="supervisor")`.

```
# src/contentforge/agents/research_agent.py
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langgraph.types import Command

from contentforge.models.llm import get_model
from contentforge.state import TeamState
from contentforge.tools.lookup import lookup_facts
from contentforge.utils.logger import get_logger
from contentforge.utils.prompts import load_prompt

def run_research(topic: str) -> str:
    logger = get_logger(__name__)
    # why: look things up FIRST, with a tool — the notes can only be as
    # true as what was found
    facts = lookup_facts.invoke({"topic": topic})
    if facts == "":
        # when: nothing known about the topic — say so, never invent
        logger.warning("no facts found for %r", topic)
        return ""
    prompt = ChatPromptTemplate.from_template(load_prompt("research"))
    chain = prompt | get_model() | StrOutputParser()
    return chain.invoke({"topic": topic, "facts": facts})

# new in Step 5: the node wrapper
def research_node(state: TeamState) -> Command:
    # why: research_done is set even when nothing was found, so the
```

```
    # Supervisor never sends the same lookup twice (Doc11's guard)
    notes = run_research(state["topic"])
    return Command(update={"research_notes": notes, "research_done": True},
                   goto="supervisor")
```

```
# src/contentforge/agents/analysis_agent.py
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langgraph.types import Command

from contentforge.models.llm import get_model
from contentforge.state import TeamState
from contentforge.utils.prompts import load_prompt

def run_analysis(notes: str) -> str:
    # why: a plain chain, no tool — organizing is thinking, not looking up
    prompt = ChatPromptTemplate.from_template(load_prompt("analysis"))
    chain = prompt | get_model() | StrOutputParser()
    return chain.invoke({"notes": notes})

# new in Step 5: the node wrapper
def analysis_node(state: TeamState) -> Command:
    findings = run_analysis(state["research_notes"])
    return Command(update={"findings": findings}, goto="supervisor")
```

```
# src/contentforge/agents/writer_agent.py
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langgraph.types import Command

from contentforge.models.llm import get_model
from contentforge.state import TeamState
from contentforge.utils.prompts import load_prompt

def build_writer_chain():
    # why: the Writer's whole skill is one LCEL chain (Doc05) — proven
    # alone in Step 1, and never rewritten after that
    prompt = ChatPromptTemplate.from_template(load_prompt("writer"))
    return prompt | get_model() | StrOutputParser()

def run_writer(findings: str, feedback: str = "") -> str:
    # why: feedback is what turns a one-shot chain into an agent that
    # can improve its own work
    if feedback == "":
        feedback = "none"
```

```
    chain = build_writer_chain()
    return chain.invoke({"findings": findings, "feedback": feedback})

# new in Step 5: the node wrapper
def writer_node(state: TeamState) -> Command:
    # how: on a revision, the newest Reviewer feedback goes back in
    feedback = ""
    if state["feedback_history"]:
        feedback = state["feedback_history"][-1]
    draft = run_writer(state["findings"], feedback)
    return Command(update={"draft": draft}, goto="supervisor")
```

Story — `reviewer_node`: Step 2's loop logic, now as a node. On a pass it sets `accepted`. On a fail it adds the feedback to the history and counts the revision — and while revisions are left, it **clears the draft**, which is what makes the Supervisor send the work back to the Writer (Doc11). `MAX_REVISIONS` now lives in the Supervisor, the one place that stops the loop. **If not:** the loop would have no memory of past feedback, or no way back to the Writer.

```
# src/contentforge/agents/reviewer_agent.py
from langchain_core.prompts import ChatPromptTemplate
from langgraph.types import Command

from contentforge.agents.supervisor import MAX_REVISIONS
from contentforge.models.llm import get_model
from contentforge.schemas import ReviewVerdict
from contentforge.state import TeamState
from contentforge.utils.logger import get_logger
from contentforge.utils.prompts import load_prompt

# why: the pass mark is decided in code, not by the model's mood — so
# the bar is the same on every run and easy to change in one place
PASS_SCORE = 7

def run_reviewer(findings: str, draft: str) -> ReviewVerdict:
    prompt = ChatPromptTemplate.from_template(load_prompt("reviewer"))
    # how: with_structured_output returns a checked ReviewVerdict (Doc05)
    chain = prompt | get_model().with_structured_output(ReviewVerdict)
    return chain.invoke({"findings": findings, "draft": draft})

# new in Step 5: Step 2's loop, as a node
def reviewer_node(state: TeamState) -> Command:
    logger = get_logger(__name__)
    verdict = run_reviewer(state["findings"], state["draft"])
    logger.info("review score %s: %s", verdict.score, verdict.feedback)

    if verdict.score >= PASS_SCORE:
```

```
        return Command(update={"accepted": True}, goto="supervisor")

    new_count = state["revision_count"] + 1
    # why: keep EVERY rejection reason (Doc11), so a "couldn't agree"
    # report shows the whole pattern, not just the last one
    history = state["feedback_history"] + [verdict.feedback]
    update = {"revision_count": new_count, "feedback_history": history}
    # how: under the limit, clear the draft so the Supervisor sends the
    # work back to the Writer; at the limit, keep the last draft
    if new_count < MAX_REVISIONS:
        update["draft"] = ""
    return Command(update=update, goto="supervisor")
```

Story — `graph.py`: five nodes and **one** edge (`START → supervisor`). Every other move is a `Command(goto=...)` returned at run time. **If not:** you'd draw fixed edges, and the Supervisor would have nothing to decide.

```
# src/contentforge/graph.py
from langgraph.graph import START, StateGraph

from contentforge.agents.analysis_agent import analysis_node
from contentforge.agents.research_agent import research_node
from contentforge.agents.reviewer_agent import reviewer_node
from contentforge.agents.supervisor import supervisor
from contentforge.agents.writer_agent import writer_node
from contentforge.state import TeamState

def build_graph():
    builder = StateGraph(TeamState)
    builder.add_node("supervisor", supervisor)
    builder.add_node("research_agent", research_node)
    builder.add_node("analysis_agent", analysis_node)
    builder.add_node("writer_agent", writer_node)
    builder.add_node("reviewer_agent", reviewer_node)
    # how: only ONE edge is drawn — every other move is a Command(goto=...)
    # returned by a node, so the Supervisor decides the route at run time
    builder.add_edge(START, "supervisor")
    return builder.compile()
```

Changed from Step 4: `print_report()` keeps its name and input — it only **gains** two things: the routing log at the top, and a "nothing is known" branch (the team can now stop right after an empty research). `main()` runs the graph instead of calling the agents one by one. And `write_with_review()` is removed: its job — write, review, send back, stop at the limit — is now done by the Supervisor and `reviewer_node`. That is the one function in the project that gets replaced, because the whole point of this step is handing that loop to the team.

```
# src/contentforge/main.py
import sys

import openai

from contentforge.config import load_config
from contentforge.exceptions import MissingConfigError
from contentforge.graph import build_graph
from contentforge.state import start_state
from contentforge.utils.logger import get_logger

def print_report(result: dict) -> None:
    # new in Step 5: the routing log
    print("Routing log:")
    for entry in result["routing_log"]:
        print(f"  {entry['step']}: {entry['reason']}")
    if result["accepted"]:
        print("\nFinal article:\n")
        print(result["draft"])
    # new in Step 5: the team can stop after an empty research
    elif result["research_notes"] == "":
        print("\nNo article: nothing is known about this topic yet.")
    else:
        # why: never pass a rejected draft off as finished
        attempts = result["revision_count"]
        print(f"\nCouldn't agree after {attempts} revisions. Feedback:")
        number = 1
        for feedback in result["feedback_history"]:
            print(f"  round {number}: {feedback}")
            number = number + 1
        print("\nLast draft (NOT approved):\n")
        print(result["draft"])

def main() -> None:
    try:
        load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    logger = get_logger(__name__)
    graph = build_graph()
    topic = input("Topic: ").strip()
    try:
        result = graph.invoke(start_state(topic))
    except openai.AuthenticationError:
        logger.error("Authentication failed — check OPENAI_API_KEY.")
```

```
        print("Login failed — check OPENAI_API_KEY in your .env file.")
        sys.exit(1)
    print_report(result)

if __name__ == "__main__":
    main()
```

Story — `tests/unit/test_supervisor.py`: the routing rules on hand-made states — every step in order, and every way the run can stop. No model, no graph, no key. **If not:** a wrong rule order would only show up as a strange, expensive run.

```
# tests/unit/test_supervisor.py
# Checks the Supervisor's routing rules with hand-made states — no model,
# no graph, no API key.
# Run from the repo root: python tests/unit/test_supervisor.py
from contentforge.agents.supervisor import MAX_REVISIONS, decide_next
from contentforge.state import start_state

def show(label: str, state: dict) -> None:
    step, reason = decide_next(state)
    print(f"{label} -> {step} ({reason})")

def test_routing_order() -> None:
    state = start_state("electric bikes")
    show("case 1 (fresh start)", state)

    state["research_done"] = True
    state["research_notes"] = "notes"
    show("case 2 (research done)", state)

    state["findings"] = "findings"
    show("case 3 (findings ready)", state)

    state["draft"] = "draft"
    show("case 4 (draft ready)", state)

def test_stopping_rules() -> None:
    nothing_found = start_state("deep sea mining")
    nothing_found["research_done"] = True
    show("case 5 (nothing found)", nothing_found)

    revising = start_state("electric bikes")
    revising["research_done"] = True
    revising["research_notes"] = "notes"
    revising["findings"] = "findings"
    revising["feedback_history"] = ["add sources"]
```

```
    revising["revision_count"] = 1
    show("case 6 (draft rejected)", revising)

    revising["revision_count"] = MAX_REVISIONS
    revising["draft"] = "last draft"
    show("case 7 (limit reached)", revising)

    revising["revision_count"] = 1
    revising["accepted"] = True
    show("case 8 (accepted)", revising)

if __name__ == "__main__":
    test_routing_order()
    test_stopping_rules()
```

Changed from Step 4: the integration test now runs the **whole team**: the weak-draft check, a full pipeline, and an unknown topic.

```
# tests/integration/test_live_agents.py
# Runs the agents and the whole team against the real OpenAI API.
# Run from the repo root: python tests/integration/test_live_agents.py
# Makes real API calls — needs a valid OPENAI_API_KEY in .env.
from contentforge.agents.reviewer_agent import PASS_SCORE, run_reviewer
from contentforge.config import load_config
from contentforge.graph import build_graph
from contentforge.state import start_state

FINDINGS = (
    "Range:\n- A battery gives 40 to 100 km per charge. [battery-review]\n"
    "Rules:\n- EU e-bike motors are limited to 25 km/h. [eu-rules]\n"
)

def test_reviewer_rejects_weak_draft() -> None:
    # case 1: a weak draft must fail — a Reviewer that passes everything
    # would never make the revision loop run
    verdict = run_reviewer(FINDINGS, "E-bikes are good.")
    passed = verdict.score >= PASS_SCORE
    print(f"case 1 (weak draft)     -> score {verdict.score}, "
          f"passed={passed}")

def test_full_pipeline(graph) -> None:
    # case 2: a known topic goes through all four specialists
    result = graph.invoke(start_state("electric bikes"))
    steps = []
    for entry in result["routing_log"]:
        steps.append(entry["step"])
```

```
    print(f"case 2 (full pipeline)  -> {steps}")
    print(f"                           accepted={result['accepted']}, "
          f"revisions={result['revision_count']}")

def test_unknown_topic(graph) -> None:
    # case 3: nothing is known — the team must stop, not invent facts
    result = graph.invoke(start_state("deep sea mining"))
    last = result["routing_log"][-1]
    print(f"case 3 (unknown topic)  -> {last['step']}: {last['reason']}")

if __name__ == "__main__":
    load_config()
    graph = build_graph()
    test_reviewer_rejects_weak_draft()
    test_full_pipeline(graph)
    test_unknown_topic(graph)
```

Run everything:

```
python tests/unit/test_lookup.py
python tests/unit/test_supervisor.py
python tests/integration/test_live_agents.py
python -m contentforge.main
```

Expected unit output:

```
case 1 (fresh start) -> research_agent (no research yet)
case 2 (research done) -> analysis_agent (research done, no findings yet)
case 3 (findings ready) -> writer_agent (findings ready, no draft yet)
case 4 (draft ready) -> reviewer_agent (draft ready for review)
case 5 (nothing found) -> END (research found no facts for this topic)
case 6 (draft rejected) -> writer_agent (revising after reviewer feedback)
case 7 (limit reached) -> END (revision limit (3) reached)
case 8 (accepted) -> END (reviewer accepted the draft)
```

Expected app run (log lines left out; the article varies):

```
Topic: electric bikes
Routing log:
  research_agent: no research yet
  analysis_agent: research done, no findings yet
  writer_agent: findings ready, no draft yet
  reviewer_agent: draft ready for review
  writer_agent: revising after reviewer feedback
  reviewer_agent: draft ready for review
```

```
  END: reviewer accepted the draft

Final article:

E-Bikes: How Far, How Fast, and Why Europe Loves Them
...
```